Raspberry Pi hardware documentation (selected chapters)

Power supply

The power supply requirements differ by Raspberry Pi model. All models require a 5.1V supply, but the current required generally increases according to model. All models up to the Raspberry Pi 3 require a micro USB power connector, while Raspberry Pi 4, Raspberry Pi 400, and Raspberry Pi 5 use a USB-C connector.

The current consumed by each Raspberry Pi depends on the peripherals connected.

Recommended power supplies

For Raspberry Pi 1, Raspberry Pi 2, and Raspberry Pi 3, we recommend the 2.5A micro USB supply. For Raspberry Pi 4 and Raspberry Pi 400, we recommend the 3A USB-C Supply for Raspberry Pi 4. For Raspberry Pi 5, we recommend the 27 W USB-C Power Supply.

NOTE: No Raspberry Pi models support USB-PPS.

NOTE: If you use a third-party USB-PD multi-port power supply, plugging an additional device into the supply when your Raspberry Pi is connected causes a renegotiation between the supply and the Raspberry Pi. If the Raspberry Pi is powered, this happens seamlessly. If the Raspberry Pi is powered down, this renegotiation can cause the Raspberry Pi to boot.

Power over Ethernet (PoE) connector

Raspberry Pi 5 PoE header

The Ethernet jack on Raspberry Pi 5 is PoE+ capable, supporting the IEEE 802.3at-2009 PoE standard.

The Ethernet jack on Raspberry Pi 4B and 3B+ is PoE capable, supporting the IEEE 802.3af-2003 PoE standard.

All Raspberry Pi models with a PoE-capable Ethernet jack require a HAT to draw power through the Ethernet port. For models that support PoE, we recommend the PoE HAT. For models that support PoE+, we recommend the PoE+ HAT.

Typical power requirements

Product | Recommended PSU current capacity | Maximum total USB peripheral current draw | Typical bare-board active current consumption
Raspberry Pi 1 Model A | 700mA | 500mA | 200mA
Raspberry Pi 1 Model B | 1.2A | 500mA | 500mA
Raspberry Pi 1 Model A+ | 700mA | 500mA | 180mA
Raspberry Pi 1 Model B+ | 1.8A | 1.2A | 330mA
Raspberry Pi 2 Model B | 1.8A | 1.2A | 350mA
Raspberry Pi 3 Model B | 2.5A | 1.2A | 400mA
Raspberry Pi 3 Model A+ | 2.5A | Limited by PSU, board, and connector ratings only. | 350mA
Raspberry Pi 3 Model B+ | 2.5A | 1.2A | 500mA
Raspberry Pi 4 Model B | 3.0A | 1.2A | 600mA
Raspberry Pi 5 | 5.0A | 1.6A (600mA if using a 3A power supply) | 800mA
Pi 400 | 3.0A | 1.2A | 800mA
Pi 500 | 5.0A | 1.6A (600mA if using a 3A power supply) | 800mA

Pi 500+ | 5.0A | 1.6A | 800mA (with LEDs off and SSD inactive)
Zero | 1.2A | Limited by PSU, board, and connector ratings only | 100mA
Zero W | 1.2A | Limited by PSU, board, and connector ratings only. | 150mA
Zero 2 W | 2A | Limited by PSU, board, and connector ratings only. | 350mA

[NOTE]

Raspberry Pi 5 provides 1.6 A of power to downstream USB peripherals when connected to a power supply capable of 5 A at +5 V (25 W). When connected to any other compatible power supply, Raspberry Pi 5 restricts downstream USB devices to 600 mA of power.

The power requirements of a Raspberry Pi increase as you use its interfaces and attach peripherals to it.

Most Raspberry Pi computers provide enough current to power standard USB devices such as keyboards, mice, and basic adapters. However, high-demand devices — including modems, external drives, and high-powered antennae — require additional power.

To check the current status of power output to the USB ports on your Raspberry Pi 5, run:

$ vcgencmd get_config usb_max_current_enable

Before connecting peripherals, check their power ratings and choose a power supply accordingly. If you're unsure whether your power supply can deliver sufficient current, connect high-power devices using an externally powered USB hub. For more information, see USB accessories.

The following table lists typical power requirements for common onboard interfaces and official accessories:

Interface or peripheral | Typical current draw
GPIO pins (combined) | 50 mA maximum
GPIO pin (individual) | Up to 16 mA
HDMI port | 50 mA
Camera Module | 250 mA
Case fan | 60 mA at 5 V (300 mW)
USB keyboard or mouse | 100 mA to 1,000 mA

The following table describes the current (in amps) drawn by different Raspberry Pi models across various workloads:

| | Raspberry Pi 1B+ | Raspberry Pi 2B | Raspberry Pi 3B | Raspberry Pi Zero | Raspberry Pi 4B
Boot | Max | 0.26 | 0.40 | 0.75 | 0.20 | 0.85
| Avg | 0.22 | 0.22 | 0.35 | 0.15 | 0.7
Idle | Avg | 0.20 | 0.22 | 0.30 | 0.10 | 0.6
Video playback (H.264) | Max | 0.30 | 0.36 | 0.55 | 0.23 | 0.85
| Avg | 0.22 | 0.28 | 0.33 | 0.16 | 0.78
Stress | Max | 0.35 | 0.82 | 1.34 | 0.35 | 1.25
| Avg | 0.32 | 0.75 | 0.85 | 0.23 | 1.2
Halt current | | | | 0.10 | 0.055 | 0.023

[NOTE]

These measurements are based on a standard Raspberry Pi OS image (current as of 26 February 2016, or June 2019 for Raspberry Pi 4), at room temperature, with the Raspberry Pi connected to an HDMI monitor, USB keyboard, and USB mouse. Raspberry Pi 3 Model B was connected to a wireless LAN access point, and Raspberry Pi 4 was connected to Ethernet. All power measurements are approximate and don't include power drawn by additional USB devices; total power consumption can exceed these figures if additional USB devices or a HAT are attached.

[.whitepaper, title="Extra PMIC features on Raspberry Pi 4, Raspberry Pi 5 and Compute Module 4", subtitle="", link=https://pip.raspberrypi.com/documents/RP-004340-WP] A range of PMIC devices have been used on Raspberry Pi 4, Raspberry Pi 5, and CM4. All the PMICs provide extra functionality alongside that of voltage supply. This document describes how to access these features in software.

Decrease Raspberry Pi 5 wattage when turned off

By default, the Raspberry Pi 5 consumes around 1 W to 1.4 W of power when turned off. This can be decreased by manually editing the EEPROM configuration with sudo rpi-eeprom-config -e. Modify the settings to add:

POWER_OFF_ON_HALT=1

This should drop the power consumption when powered down to around 0.01 W.

Power supply warnings

On all models of Raspberry Pi since the Raspberry Pi B+ (2014) except the Zero range, there is low-voltage detection circuitry that will detect if the supply voltage drops below 4.63 V (±5%). This will result in an entry being added to the kernel log.

If you see warnings, switch to a higher quality power supply and cable. Low quality power supplies can corrupt storage or cause unpredictable behaviour within the Raspberry Pi.

Voltages can drop for a variety of reasons. You might have plugged in too many high-demand USB devices. The power supply could be inadequate. Or the power supply cable could use wires that are too thin.

[.whitepaper, title="Making a more resilient file system", subtitle="", link=https://pip.raspberrypi.com/documents/RP-003610-WP] Raspberry Pi devices are frequently used as data storage and monitoring devices, often in places where sudden power-downs can occur. As with any computing device, power dropouts can cause storage corruption.

This white paper provides some options on how to prevent data corruption under these and other circumstances by selecting appropriate file systems and setups to ensure data integrity.

Power supplies and Raspberry Pi OS

The bootloader passes information about the power supply through device-tree /proc/device-tree/chosen/power. Users will typically not read this directly.

max_current:: The max current in mA uspd_power_data_objects:: A dump of the PDOs – debug for advanced users usb_max_current_enable:: Whether the current limiter was set to high or low usb_over_current_detected:: Whether any USB over current occurred during boot before

transferring control to the OS reset_event:: The PMIC reset reason, for example, watchdog, over- or under-voltage, over-temperature

The PMIC has built-in ADCs that, among other things, can measure the supply voltage EXT5V_V. Use the following command to view ADC measurements:

```
$ vcgencmd pmic_read_adc
```

You can't see USB current or anything else connected directly to 5V, because this bypasses the PMIC. You should not expect this to add up to the wattage of the source power supply. However, it can be useful to monitor things like the core voltage.

Back-powering

The USB specification requires that USB devices must not supply current to upstream devices. If a USB device does supply current to an upstream device, then this is called back-powering. Often this happens when a badly-made powered USB hub is connected, and will result in the powered USB hub supplying power to the host Raspberry Pi. We don't recommended this because power supplied to the Raspberry Pi through the hub bypasses the protection circuitry built into the Raspberry Pi, leaving it vulnerable to damage in the event of a power surge.

Raspberry Pi revision codes

Each distinct Raspberry Pi model revision has a unique revision code. You can look up a Raspberry Pi device's revision code by running:

```
$ cat /proc/cpuinfo
```

The last three lines show the hardware type, the revision code, and the Raspberry Pi device's unique serial number. For example:

```
Hardware    : BCM2835 Revision    : a02082 Serial      : 00000000765fc593
```

NOTE: All Raspberry Pi computers report BCM2835, even those with BCM2836, BCM2837, BCM2711, and BCM2712 processors. You should not use this string to detect the processor. Decode the revision code using the information below, or cat /sys/firmware/devicetree/base/model. Depending on which kernel you're running, your cpuinfo might also have a "Model" line, and might not have a "Hardware" line.

Old-style revision codes

The first set of Raspberry Pi models were given sequential hex revision codes from 0002 to 0015:

Code	Model	Revision	RAM	Manufacturer
0002	B	1.0	256 MB	Egoman
0003	B	1.0	256 MB	Egoman
0004	B	2.0	256 MB	Sony UK
0005	B	2.0	256 MB	Qisda
0006	B	2.0	256 MB	Egoman
0007	A	2.0	256 MB	Egoman
0008	A	2.0	256 MB	Sony UK
0009	A	2.0	256 MB	Qisda

000d | B | 2.0 | 512 MB | Egoman
000e | B | 2.0 | 512 MB | Sony UK
000f | B | 2.0 | 512 MB | Egoman
0010 | B+ | 1.2 | 512 MB | Sony UK
0011 | CM1 | 1.0 | 512 MB | Sony UK
0012 | A+ | 1.1 | 256 MB | Sony UK
0013 | B+ | 1.2 | 512 MB | Embest
0014 | CM1 | 1.0 | 512 MB | Embest
0015 | A+ | 1.1 | 256 MB / 512 MB | Embest

New-style revision codes

With the launch of the Raspberry Pi 2, new-style revision codes were introduced. Rather than being sequential, each bit of the hex code represents a piece of information about the revision:

NOQuuuWuFMMMCCCCPPPPTTTTTTTTRRRR

Part | Represents | Options
N (bit 31) | Overvoltage | 0: Overvoltage allowed
| | 1: Overvoltage disallowed
O (bit 30) | OTP Program^1^ | 0: OTP programming allowed
| | 1: OTP programming disallowed
Q (bit 29) | OTP Read^1^ | 0: OTP reading allowed
| | 1: OTP reading disallowed
uuu (bits 26-28) | Unused | Unused
W (bit 25) | Warranty bit^2^ | 0: Warranty is intact
| | 1: Warranty has been voided by overclocking
u (bit 24) | Unused | Unused
F (bit 23) | New flag | 1: new-style revision
| | 0: old-style revision
MMM (bits 20-22) | Memory size | 0: 256 MB
| | 1: 512 MB
| | 2: 1 GB
| | 3: 2 GB
| | 4: 4 GB
| | 5: 8 GB
| | 6: 16 GB
| | 7: Other^3^
CCCC (bits 16-19) | Manufacturer | 0: Sony UK
| | 1: Egoman
| | 2: Embest
| | 3: Sony Japan
| | 4: Embest
| | 5: Stadium
PPPP (bits 12-15) | Processor | 0: BCM2835
| | 1: BCM2836
| | 2: BCM2837
| | 3: BCM2711
| | 4: BCM2712
TTTTTTTT (bits 4-11) | Type | 0x00: A
| | 0x01: B
| | 0x02: A+

| | 0x03: B+
| | 0x04: 2B
| | 0x05: Alpha (early prototype)
| | 0x06: CM1
| | 0x08: 3B
| | 0x09: Zero
| | 0x0a: CM3
| | 0x0c: Zero W
| | 0x0d: 3B+
| | 0x0e: 3A+
| | 0x0f: Internal use only
| | 0x10: CM3+
| | 0x11: 4B
| | 0x12: Zero 2 W
| | 0x13: 400
| | 0x14: CM4
| | 0x15: CM4S
| | 0x16: Internal use only
| | 0x17: 5
| | 0x18: CM5
| | 0x19: 500/500+
| | 0x1a: CM5 Lite
| | 0x1b: CM0
RRRR (bits 0-3) | Revision | 0, 1, 2, etc.

^1^ Information on programming the OTP bits.

^2^ The warranty bit is never set on Raspberry Pi 4.

^3^ For a memory size of "Other", the actual memory capacity can be determined from the rpi-sdram-size-gbit firmware parameter.

New-style revision codes in use

NOTE: This list isn't exhaustive; there might be codes in use that aren't in this table. For best practices on using revision codes to identify boards, see the following section.

// This table is now sorted by Type (from table above), then Revision, then RAM, and finally Code. This is the most likely order-of-manufacture, which means we'll normally just add new revision-codes to the very bottom of the table, without having to worry about re-ordering entries.

Code | Model | Revision | RAM | Manufacturer
900021 | A+ | 1.1 | 512 MB | Sony UK
900032 | B+ | 1.2 | 512 MB | Sony UK
a01040 | 2B | 1.0 | 1 GB | Sony UK
a01041 | 2B | 1.1 | 1 GB | Sony UK
a21041 | 2B | 1.1 | 1 GB | Embest
a02042 | 2B (with BCM2837) | 1.2 | 1 GB | Sony UK
a22042 | 2B (with BCM2837) | 1.2 | 1 GB | Embest
900061 | CM1 | 1.1 | 512 MB | Sony UK
a02082 | 3B | 1.2 | 1 GB | Sony UK

a22082 | 3B | 1.2 | 1 GB | Embest
a32082 | 3B | 1.2 | 1 GB | Sony Japan
a52082 | 3B | 1.2 | 1 GB | Stadium
a22083 | 3B | 1.3 | 1 GB | Embest
900092 | Zero | 1.2 | 512 MB | Sony UK
920092 | Zero | 1.2 | 512 MB | Embest
900093 | Zero | 1.3 | 512 MB | Sony UK
920093 | Zero | 1.3 | 512 MB | Embest
a020a0 | CM3 | 1.0 | 1 GB | Sony UK
a220a0 | CM3 | 1.0 | 1 GB | Embest
9000c1 | Zero W | 1.1 | 512 MB | Sony UK
a020d3 | 3B+ | 1.3 | 1 GB | Sony UK
a020d4 | 3B+ | 1.4 | 1 GB | Sony UK
9020e0 | 3A+ | 1.0 | 512 MB | Sony UK
9020e1 | 3A+ | 1.1 | 512 MB | Sony UK
a02100 | CM3+ | 1.0 | 1 GB | Sony UK
a03111 | 4B | 1.1 | 1 GB | Sony UK
b03111 | 4B | 1.1 | 2 GB | Sony UK
c03111 | 4B | 1.1 | 4 GB | Sony UK
b03112 | 4B | 1.2 | 2 GB | Sony UK
c03112 | 4B | 1.2 | 4 GB | Sony UK
b03114 | 4B | 1.4 | 2 GB | Sony UK
c03114 | 4B | 1.4 | 4 GB | Sony UK
d03114 | 4B | 1.4 | 8 GB | Sony UK
b03115 | 4B | 1.5 | 2 GB | Sony UK
c03115 | 4B | 1.5 | 4 GB | Sony UK
d03115 | 4B | 1.5 | 8 GB | Sony UK
902120 | Zero 2 W | 1.0 | 512 MB | Sony UK
c03130 | 400 | 1.0 | 4 GB | Sony UK
a03140 | CM4 | 1.0 | 1 GB | Sony UK
b03140 | CM4 | 1.0 | 2 GB | Sony UK
c03140 | CM4 | 1.0 | 4 GB | Sony UK
d03140 | CM4 | 1.0 | 8 GB | Sony UK
b04170 | 5 | 1.0 | 2 GB | Sony UK
c04170 | 5 | 1.0 | 4 GB | Sony UK
d04170 | 5 | 1.0 | 8 GB | Sony UK
a04171 | 5 | 1.1 | 1 GB | Sony UK
b04171 | 5 | 1.1 | 2 GB | Sony UK
c04171 | 5 | 1.1 | 4 GB | Sony UK
d04171 | 5 | 1.1 | 8 GB | Sony UK
e04171 | 5 | 1.1 | 16 GB | Sony UK
b04180 | CM5 | 1.0 | 2 GB | Sony UK
c04180 | CM5 | 1.0 | 4 GB | Sony UK
d04180 | CM5 | 1.0 | 8 GB | Sony UK
e04180 | CM5 | 1.0 | 16 GB | Sony UK
d04190 | 500 | 1.0 | 8 GB | Sony UK
e04190 | 500+ | 1.0 | 16 GB | Sony UK
b041a0 | CM5 Lite | 1.0 | 2 GB | Sony UK
c041a0 | CM5 Lite | 1.0 | 4 GB | Sony UK
d041a0 | CM5 Lite | 1.0 | 8 GB | Sony UK
e041a0 | CM5 Lite | 1.0 | 16 GB | Sony UK

Using revision codes for board identification

From the command line we can use the following to get the revision code of the board:

```
$ cat /proc/cpuinfo | grep Revision
Revision      : c03111
```

In this example above, we have a hexadecimal revision code of c03111. Converting this to binary, we get 0 0 0 000 0 0 1 100 0000 0011 00010001 0001. Spaces have been inserted to show the borders between each section of the revision code, according to the above table.

Starting from the lowest order bits, the bottom four (0-3) are the board revision number, so this board has a revision of 1. The next eight bits (4-11) are the board type, in this case binary 00010001, hex 11, so this is a Raspberry Pi 4B. Using the same process, we can determine that the processor is a BCM2711, the board was manufactured by Sony UK, and it has 4 GB of RAM.

Getting the revision code in your program

Obviously there are so many programming languages out there it's not possible to give examples for all of them, but here are two quick examples for C and Python. Both these examples use a system call to run a bash command that gets the cpuinfo and pipes the result to awk to recover the required revision code. They then use bit operations to extract the New, Model, and Memory fields from the code.

```
#include <stdio.h> #include <stdlib.h>

int main( int argc, char argv[] ) { FILE fp; char revcode[32];

fp = popen("cat /proc/cpuinfo | awk '/Revision/ {print $3}'", "r");
if (fp == NULL)
exit(1);
fgets(revcode, sizeof(revcode), fp);
pclose(fp);

int code = strtol(revcode, NULL, 16); int new = (code >> 23) & 0x1; int model = (code >> 4) &
0xff; int mem = (code >> 20) & 0x7;

if (new && model == 0x11 && mem >= 3)  // Note, 3 in the mem field is 2 GB printf("We are a 4B
with at least 2 GB of RAM!\n" );

return 0; }
```

And the same in Python:

```
import subprocess

cmd = "cat /proc/cpuinfo | awk '/Revision/ {print $3}'"
revcode = subprocess.check_output(cmd, shell=True)

code = int(revcode, 16) new = (code >> 23) & 0x1 model = (code >> 4) & 0xff mem = (code >> 20) &
0x7
```

if new and model == 0x11 and mem >= 3 : # Note, 3 in the mem field is 2 GB print("We are a 4B with at least 2 GB RAM!")

Best practices for revision code usage

To avoid problems when new board revisions are created, do not use the revision code (for example, c03111).

A naive implementation uses a list of supported revision codes, comparing the detected code with the list to decide if the device is supported. This breaks when a new board revision comes out or if the production location changes: each creates a new revision code not in the supported revision code list. This would cause rejections of new revisions of the same board type, despite the fact that they are always backwards-compatible. Every time a new revision appears, you would have to release a new supported revision code list containing the new revision code – an onerous support burden.

Instead, use one of the following approaches:

Filter by the board-type field (3A, 4B, etc.), which indicates the model, but not the revision.
Filter by the amount-of-memory field, since RAM vaguely corresponds to the computing power of a board.

For instance, you could limit support to Raspberry Pi 4B models with 2 GB of RAM or more. The examples in the previous section use this recommended approach.

NOTE: Always check bit 23, the 'New' flag, to ensure that the revision code is the new version before checking any other fields.

Check Raspberry Pi model and CPU across distributions

Support and formatting for /proc/cpuinfo varies across Linux distributions. To check the model or CPU of a Raspberry Pi device on any Linux distribution (including Raspberry Pi OS), check the device tree:

```
$ cat /proc/device-tree/compatible | tr '\0' '\n'
raspberrypi,5-model-b
brcm,bcm2712
```

This outputs two null-separated string values, each containing a comma-separated make and model. For instance, the Raspberry Pi 5 outputs the board and CPU strings above. These correspond to the following values:

raspberrypi (board make) 5-model-b (board model) brcm (CPU make) bcm2712 (CPU model)

Raspberry Pi models have the following device tree values:

Device Name	Make	Model	CPU Make	CPU
Raspberry Pi 500 /500+	raspberrypi	500	brcm	bcm2712
Compute Module 5	raspberrypi	5-compute-module	brcm	bcm2712
Raspberry Pi 5	raspberrypi	5-model-b	brcm	bcm2712
Raspberry Pi 400	raspberrypi	400	brcm	bcm2711
Compute Module 4S	raspberrypi	4s-compute-module	brcm	bcm2711

Compute Module 4 | raspberrypi | 4-compute-module | brcm | bcm2711
Raspberry Pi 4 Model B | raspberrypi | 4-model-b | brcm | bcm2711
Raspberry Pi Zero 2 W | raspberrypi | model-zero-2-w | brcm | bcm2837
Compute Module 3+ | raspberrypi | 3-plus-compute-module | brcm | bcm2837
Compute Module 3 | raspberrypi | 3-compute-module | brcm | bcm2837
Raspberry Pi 3 Model A+ | raspberrypi | 3-model-a-plus | brcm | bcm2837
Raspberry Pi 3 Model B+ | raspberrypi | 3-model-b-plus | brcm | bcm2837
Raspberry Pi 3 Model B | raspberrypi | 3-model-b | brcm | bcm2837
Raspberry Pi 2 Model B Rev 1.2 | raspberrypi | 2-model-b-rev2 | brcm | bcm2837
Raspberry Pi 2 Model B | raspberrypi | 2-model-b | brcm | bcm2836
Raspberry Pi Zero W | raspberrypi | model-zero-w | brcm | bcm2835
Raspberry Pi Zero | raspberrypi | model-zero | brcm | bcm2835
Compute Module 1 | raspberrypi | compute-module | brcm | bcm2835
Raspberry Pi Model A+ | raspberrypi | model-a-plus | brcm | bcm2835
Raspberry Pi Model B+ | raspberrypi | model-b-plus | brcm | bcm2835
Raspberry Pi Model B Rev 2 | raspberrypi | model-b-rev2 | brcm | bcm2835
Raspberry Pi Model A | raspberrypi | model-a | brcm | bcm2835
Raspberry Pi Model B | raspberrypi | model-b | brcm | bcm2835

OTP register and bit definitions

All SoCs used by the Raspberry Pi range have a inbuilt one-time programmable (OTP) memory block. A few locations have factory-programmed data.

OTP memory size:

non-BCM2712 devices: 66 32-bit values BCM2712 devices: 192 32-bit values

To display the contents of the OTP, run the following command:

$ vcgencmd otp_dump

OTP registers on non-BCM2712 devices

This list contains the publicly available information on the registers. If a register or bit is not defined here, then it is not public.

16:: OTP control register – BCM2711 + Bit 26: disables VC JTAG Bit 27: disables VC JTAG

17:: bootmode register + Bit 1: sets the oscillator frequency to 19.2 MHz Bit 3: enables pull ups on the SDIO pins Bit 15: disables ROM RSA key 0 – (secure boot enabled if set) (BCM2711) Bit 19: enables GPIO bootmode Bit 20: sets the bank to check for GPIO bootmode Bit 21: enables booting from SD card Bit 22: sets the bank to boot from Bit 28: enables USB device booting Bit 29: enables USB host booting (ethernet and mass storage)

NOTE: On BCM2711 the bootmode is defined by the bootloader EEPROM configuration instead of OTP.

18:: copy of bootmode register 28:: serial number 29:: ~(serial number) 30:: revision code ^1^ 33:: board revision extended – the meaning depends on the board model. This is available through device-tree in /proc/device-tree/chosen/rpi-boardrev-ext and for testing purposes this OTP value can be temporarily overridden by setting board_rev_ext in config.txt. + Compute Module 4 Bit 30: Whether the Compute Module has a Wi-Fi module fitted 0 – Wi-Fi 1 – No Wi-Fi Bit 31: Whether the

Compute Module has an EMMC module fitted 0 – EMMC 1 – No EMMC (Lite) Raspberry Pi 400 Bits 0-7: The default keyboard country code used by piwiz

35 :: High 32 bits of 64-bit serial number 36-43:: customer OTP values 45:: MPG2 decode key 46:: WVC1 decode key 47-54:: SHA256 of RSA public key for secure-boot 55:: secure-boot flags (reserved for use by the bootloader) 56-63:: 256-bit device-specific private key 64-65:: MAC address; if set, system will use this in preference to the automatically generated address based on the serial number 66:: advanced boot register (not BCM2711) + Bits 0-6: GPIO for ETH_CLK output pin Bit 7: enables ETH_CLK output Bits 8-14: GPIO for LAN_RUN output pin Bit 15: enables LAN_RUN output Bit 24: extends USB HUB timeout parameter Bit 25: ETH_CLK frequency: 0 – 25 MHz 1 – 24 MHz

^1^Also contains bits to disable overvoltage, OTP programming, and OTP reading.

OTP Registers on BCM2712 devices

This list contains the publicly available information on the registers. If a register or bit is not defined here, then it is not public.

22:: bootmode register + Bit 1: Boot from SD card Bits 2-4: Booting from SPI EEPROM (and which GPIOs) Bit 10: Disable booting from SD card Bit 11: Disable booting from SPI Bit 12: Disable booting from USB

23:: copy of bootmode register 29:: advanced boot mode + Bits 0-7: GPIO for SD card detect Bits 8-15: GPIO to use for RPIBOOT

31:: lower 32 bits of serial number 32:: board revision 33:: board attributes – the meaning depends on the board model. This is available through device-tree in /proc/device-tree/chosen/rpi-boardrev-ext

35:: upper 32 bits of serial number The full 64 bit serial number is available in /proc/device-tree/serial-number

50-51:: Ethernet MAC address This is passed to the operating system in the Device Tree, for example, /proc/device-tree/axi/pcie@120000/rp1/ethernet@100000/local-mac-address

52-53:: Wi-Fi MAC address This is passed to the operating system in the Device Tree, for example, /proc/device-tree/axi/mmc@1100000/wifi@1/local-mac-address

54-55:: Bluetooth MAC address This is passed to the operating system in the Device Tree, for example, /proc/device-tree/soc/serial@7d50c000/bluetooth/local-bd-address

77-84:: customer OTP values

86:: board country – The default keyboard country code used by piwiz If set, this is available through Device Tree in /proc/device-tree/chosen/rpi-country-code

87-88:: customer Ethernet MAC address Overrides OTP rows 50-51 if set

89-90:: customer Wi-Fi MAC address Overrides OTP rows 52-53 if set

91-92:: customer Bluetooth MAC address Overrides OTP rows 54-55 if set

109-114:: Factory device UUID Currently a 16-digit numerical id which should match the bar code on the device. Padded with zero characters and c40 encoded.

This is available through device-tree in /proc/device-tree/chosen/rpi-duid.

Frequency management and thermal control

All Raspberry Pi models perform a degree of thermal management to avoid overheating under heavy load. The SoCs have an internal temperature sensor, which software on the GPU polls to ensure that temperatures do not exceed a limit which we define as 85°C on all models. It is possible to set this to a lower value, but not to a higher one. As the device approaches the limit, various frequencies and sometimes voltages used on the chip (Arm, GPU) are reduced. This reduces the amount of heat generated, keeping the temperature under control.

When the core temperature is between 80°C and 85°C, the Arm cores will be progressively throttled back. If the temperature reaches 85°C, both the Arm cores and the GPU will be throttled back.

For Raspberry Pi 3 Model B+, the PCB technology has been changed to provide better heat dissipation and increased thermal mass. In addition, a soft temperature limit has been introduced, with the goal of maximising the time for which a device can "sprint" before reaching the hard limit at 85°C. When the soft limit is reached, the clock speed is reduced from 1.4 GHz to 1.2 GHz, and the operating voltage is reduced slightly. This reduces the rate of temperature increase: we trade a short period at 1.4 GHz for a longer period at 1.2 GHz. By default, the soft limit is 60°C, and this can be changed by using the temp_soft_limit setting in config.txt.

The Raspberry Pi 4 Model B continues with the same PCB technology as the Raspberry Pi 3 Model B+, to help dissipate excess heat. There is currently no soft limit defined.

Use DVFS

NOTE: Discussion of DVFS applies to 4-series devices only (Raspberry Pi 4B, Compute Module 4, Compute Module 4S, and Raspberry Pi 400).

Raspberry Pi 4 devices implement dynamic voltage and frequency scaling (DVFS). This technique allows 4-series devices to run at lower temperatures whilst still providing the same performance.

Various clocks (for example, Arm, Core, V3D, ISP, H264, HEVC) inside the SoC are monitored by the firmware, and whenever they are not running at full speed, the voltage supplied to the particular part of the chip driven by the clock is reduced relative to the reduction from full speed. In effect, only enough voltage is supplied to keep the block running correctly at the specific speed at which it is running. This can result in significant reductions in power used by the SoC, and therefore in the overall heat being produced.

Due to possible system stability problems involved with running an undervoltage, especially when using undervoltaged fixed clock peripherals (eg. PCIe), three DVFS modes are available and can be configured in /boot/firmware/config.txt with the below properties. Most systems should use dvfs=3, headless systems might benefit from a small power reduction with dvfs=1 at the risk of PCIe stability issues.

property=value | Description
dvfs=1 | allow undervoltage
dvfs=2 | fixed voltage for default operating frequencies
dvfs=3 | scale voltage up on demand for over clocking (default). If over_voltage is specified in config.txt then dynamic voltage scaling is disabled causing the system to revert to dvfs=2.

NOTE: This setting has been removed on 5-series devices and is effectively always mode 3.

In addition, a more stepped CPU governor is also used to produce finer-grained control of Arm core frequencies, which means the DVFS is more effective. The steps are now 1500 MHz, 1000 MHz, 750 MHz, and 600 MHz. These steps can also help when the SoC is being throttled, and mean that throttling all the way back to 600 MHz is much less likely, giving an overall increase in fully loaded performance.

The default CPU governor is ondemand. The governor can be manually changed with the cpufreq-set command (from the cpufrequtils package) to reduce idle power consumption:

$ sudo apt install cpufrequtils $ sudo cpufreq-set -g powersave

Measure temperatures

Due to the architecture of the SoCs used on Raspberry Pi devices, and the use of the upstream temperature monitoring code in the Raspberry Pi OS distribution, Linux-based temperature measurements can be inaccurate. However, the vcgencmd command provides an accurate and instantaneous reading of the current SoC temperature, as it communicates with the GPU directly:

$ vcgencmd measure_temp

Add heatsinks

Thanks to built-in throttling, heatsinks are not necessary to prevent overheating damage to the SoC. However, a heatsink or small fan can reduce thermal throttling and improve performance. Mount the Raspberry Pi vertically for the best airflow and thus slightly improved heat dissipation.

Fan cases

To ensure the best performance for your Raspberry Pi, use an active cooling solution such as a fan. Raspberry Pi firmware manages fan speeds for all official fans.

Raspberry Pi 4 fan

For Raspberry Pi 4, add the Raspberry Pi 4 Case Fan to the lid of the Raspberry Pi 4 case.

The fan is rated at 0.18 A, operates at 5 V DC supplied through the 40-pin GPIO header, and typically draws 60 mA (300 mW).

Raspberry Pi 5 fans

For Raspberry Pi 5, use one of the official fan options:

Active Cooler Case for Raspberry Pi 5

Both of the Raspberry Pi 5 fan options plug into the four-pin JST-SH PWM fan connector located in the upper right of the board between the 40-pin GPIO header and the USB 2.0 ports.

The fan connector pulls from the same current limit as USB peripherals.

The official case fan draws approximately 60 mA at 5 V (300 mW).

Given the extra cooling performance of the Active Cooler, we recommend it for customers who want to overclock their Raspberry Pi 5.

The fan is scheduled to turn on and off based on the temperature of the SoC. The default temperature mapping is as follows:

Less than 50°C: the fan doesn't spin at all (0% speed). 50°C: the fan turns on at a low speed (30% speed). 60°C: the fan speed increases to a medium speed (50% speed). 67.5°C: the fan speed increases to a high speed (70% speed). 75°C: the fan increases to full speed (100% speed).

Temperature decreases use the same mapping with a 5°C hysteresis; fan speed decreases when the temperature drops to 5°C below each of the above thresholds.

NOTE: The temperature, hysteresis and speed figures given above can be adjusted by using the various fan_tempN, fan_tempN_hyst and fan_tempN_speed dtoverlay settings (where N is 0, 1, 2 or 3). See the overlays README for full details. For example, adding dtparam=fan_temp0=55000 to /boot/firmware/config.txt will cause the fan to remain off until the Raspberry Pi 5's temperature reaches 55°C.

At boot the fan is turned on, and the tachometer input is checked to see if the fan is spinning. If it is, then the cooling_fan device tree overlay is enabled. This overlay is in bcm2712-rpi-5-b.dtb by default, but with status=disabled.

Raspberry Pi 5 fan connector pinout

The Raspberry Pi 5 fan connector is a 1 mm pitch JST-SH socket containing the following four pins:

Pin | Function | Wire colour
1 | +5V | Red
2 | PWM | Blue
3 | GND | Black
4 | Tach | Yellow

Real Time Clock (RTC)

The Raspberry Pi 5 includes an RTC module. This can be battery powered through the J5 (BAT) connector on the board. When the components are facing up and the logo is in the correct orientation, the J5 connector is located towards the near left corner of the board and to the right of the USB-C power connector.

The J5 battery connector

You can set a wake alarm, which switches the board to a very low-power state (approximately 3

mA). When the alarm time is reached, the board powers back on. This can be useful for periodic jobs like time-lapse imagery.

To support the low-power mode for wake alarms:

Edit the bootloader configuration. + $ sudo rpi-eeprom-config --edit

Add the following two lines. + POWER_OFF_ON_HALT=1 WAKE_ON_GPIO=0

You can test the functionality with:

```
$ echo +600 | sudo tee /sys/class/rtc/rtc0/wakealarm
$ sudo halt
```

This command halts the board into a very low-power state, then wakes and restarts after 10 minutes.

The RTC also provides the time on boot, for example, in dmesg, for use cases that lack access to NTP:

```
[    1.295799] rpi-rtc soc:rpi_rtc: setting system clock to 2023-08-16T15:58:50 UTC (1692201530)
```

NOTE: The RTC is still usable even when there's no backup battery attached to the J5 connector.

Add a backup battery

Lithium-manganese rechargeable RTC battery

The official battery part is a rechargeable lithium manganese coin cell, with a pre-fitted two-pin JST-SH plug and an adhesive mounting pad. This is suitable for powering the RTC when the main power supply for the board is disconnected. Since the current draw when powered down measures in single-digit µA, the retention time measures in months.

NOTE: We don't recommend using a primary (non-rechargeable) lithium cell for the RTC. The RTC backup current consumption is higher than most dedicated RTC modules and results in a short service life.

WARNING: Don't use a Lithium Ion cell for the RTC.

Enable battery charging

The RTC is equipped with a constant-current (3 mA) constant-voltage charger.

Charging of the battery is disabled by default. There are sysfs files that show the charging voltage and limits:

```
$ grep . /sys/class/rtc/rtc0/charging_voltage

/sys/class/rtc/rtc0/charging_voltage:0 /sys/class/rtc/rtc0/charging_voltage_max:4400000
/sys/class/rtc/rtc0/charging_voltage_min:1300000
```

To charge the battery at a set voltage, add rtc_bbat_vchg to /boot/firmware/config.txt:

dtparam=rtc_bbat_vchg=3000000

Reboot with sudo reboot to use the new voltage setting. Check the sysfs files to ensure that the charging voltage was correctly set.

Disable battery charging

To stop charging, remove any lines that contain rtc_bbat_vchg from config.txt.

Power button

NOTE: This section only applies to Raspberry Pi models with a power button, such as the Raspberry Pi 5.

When you plug your Raspberry Pi into power for the first time, it will automatically turn on and boot into the operating system without having to push the button.

If you run Raspberry Pi Desktop, you can initiate a clean shutdown by briefly pressing the power button. A window will appear asking whether you want to shutdown, reboot, or logout.

Select an option or press the power button again to initiate a clean shutdown.

NOTE: If you run Raspberry Pi Desktop, you can press the power button twice in quick succession to shut down. If you run Raspberry Pi OS Lite without a desktop, press the power button a single time to initiate a shutdown.

Restart

If the Raspberry Pi board is turned off, but still connected to power, pressing the power button restarts the board.

NOTE: Resetting the Power Management Integrated Circuit (PMIC) can also restart the board. Connecting a HAT can reset the PMIC. Always disconnect your device from the power supply before connecting a HAT.

Hard shutdown

To force a hard shutdown, press and hold the power button.

Add your own power button

The J2 jumper

The J2 jumper is located between the RTC battery connector and the board edge. This breakout allows you to add your own power button to Raspberry Pi 5 by adding a Normally Open (NO) momentary switch bridging the two pads. Briefly closing this switch will perform the same actions as the onboard power button.